Color Recognition

1. Experiment Objective

Use OpenCV to detect specified color regions in the HSV color space, with support for ROI mouse color sampling, dynamic HSV tuning, and morphological filtering.

2. Experiment Principle

1. Key Terms

Term	Description
HSV Color Space	Hue (H, 0°~179°), Saturation (S, 0~255), Value (V, 0~255); more suitable for color threshold segmentation than RGB
ROI Color Sampling	Drag a mouse box around the target color region in the frame; the program automatically computes the HSV range of that region and generates the detection thresholds
Color Threshold (inRange)	Pixels within the threshold range in the HSV image are set to white (255) and pixels outside are set to black (0), producing a binary mask
Morphological Filtering	Opening operation (remove noise) + closing operation (fill holes) to smooth the outline of the target region
Contour Detection	Extracts connected-region contours from the binary mask to compute the area, center point, and minimum enclosing circle
Minimum Enclosing Circle	The smallest circle enclosing a contour, providing the target's center position and radius, used for drawing the detection circle

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: detect_color_node (HSV threshold segmentation → morphological
filtering → contour detection → OpenCV visualization)
        supports ROS2 dynamic parameter tuning + ROI mouse color sampling + HSV
threshold file persistence
```

Data Flow: Image subscription → image conversion → BGR→HSV → `cv2.inRange` threshold segmentation → morphological filtering → `cv2.findContours` → area/radius filtering → draw enclosing circles + labels → real-time display.

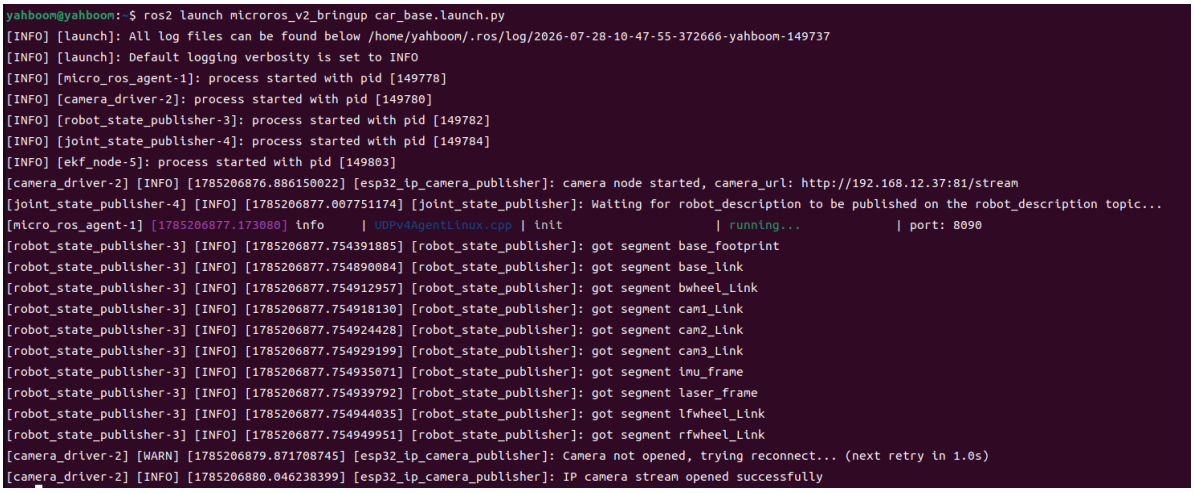
3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	☑ Supported
No-PTZ Version	☑ Supported
Required Peripherals	ESP32 camera, a colored object to detect

2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```



3. Start Color Recognition (Terminal 2)

```
ros2 launch microros_v2_opencv_visual_pkg detect_color.launch.py
```

4. Operation Instructions

Two operation modes:

Mode	How to Enter	Function
detect (detection)	Default on startup / after ROI color sampling finishes	Detects color targets in real time based on the current HSV thresholds
sample (sampling)	Press `r` / drag with the left mouse button	Box-select the target color region, compute the HSV range inside the ROI, and persist it

ROI color sampling flow: Drag a mouse box to select the region → compute the min/max of each channel inside the ROI → subtract the expansion amount for the lower bound and add it for the upper bound → S/V upper bounds are kept no lower than 253 → automatically saved to `color_hsv.txt` → switch back to detect mode.

Dynamic parameter tuning:

```
ros2 param list /detect_color_node # list all parameters currently "declared" by the /detect_color_node node
ros2 param set /detect_color_node min_target_area 100.0 # at runtime, change min_target_area to 100.0 (unit: pixel^2, i.e. the contour area)
```

Adjustable parameters: HSV upper/lower bounds, morphological kernel size, minimum area/radius, window flip/rotation, etc.

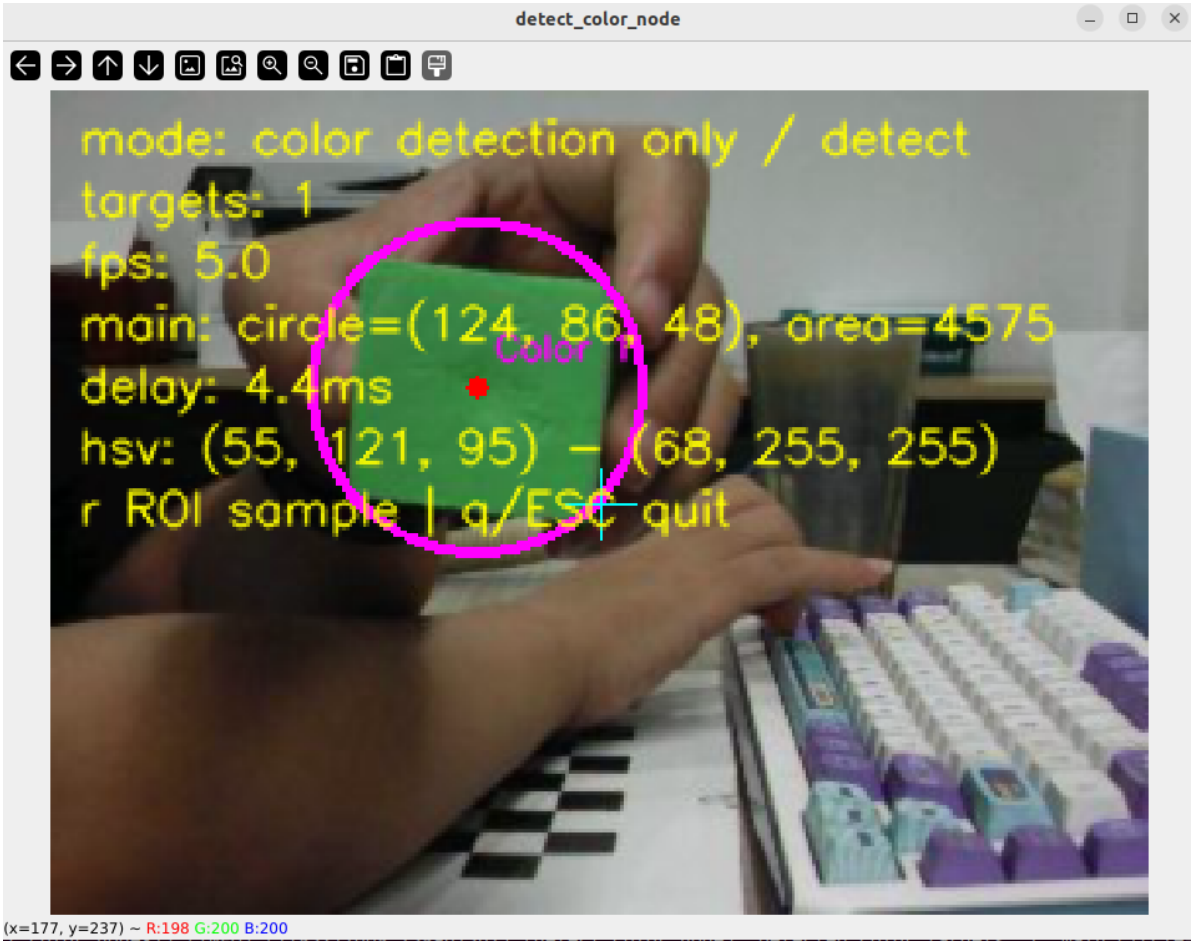
Operation	Function
Left mouse button drag	Directly box-select the target color region
`r`	Manually enter ROI color sampling mode
`q` / `ESC`	Exit

5. How to Stop

Press `Ctrl+C` to stop the terminals one by one.

4. Experiment Observations

- **Default detection (red)**: Red objects are marked with a green circle and a red center dot, labeled `Color 1`
- **Multiple targets**: Multiple objects are marked simultaneously; the one with the largest area is the purple primary target, the rest are green
- **ROI sampling switch**: After box-selecting a new color, detection switches from red to the newly selected color, and the HSV thresholds update and save automatically
- **Binary mask side by side** (`show_binary_window:=true`): the binary mask is shown side by side on the right



5. Program Analysis

> Source code paths:
>
> - Launch: `~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_color.launch.py`
> - Node: `~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_color.py`

1. Core Node Analysis

`DetectColorNode` uses HSV threshold segmentation + morphological filtering + contour detection, with support for ROI mouse color sampling and ROS2 dynamic parameter tuning.

```
# Source file:
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_color.py
class DetectColorNode(Node):
    def __init__(self):
        super().__init__('detect_color_node')
        # HSV threshold parameters (adjustable at runtime via ros2 param set)
        self.declare_parameter('Hmin', 0)
        self.declare_parameter('Hmax', 9)
        self.declare_parameter('Smin', 85)
        self.declare_parameter('Smax', 253)
        self.declare_parameter('Vmin', 126)
        self.declare_parameter('Vmax', 253)
```

```
        self.declare_parameter('min_target_area', 60.0)
        self.declare_parameter('min_target_radius', 5.0)
        self.declare_parameter('morph_kernel_size', 5)
        self.declare_parameter('show_binary_window', False)

    def detect_color_targets(self, hsv):
        lower = np.array([self.Hmin, self.Smin, self.Vmin])
        upper = np.array([self.Hmax, self.Smax, self.Vmax])
        mask = cv.inRange(hsv, lower, upper)

        # Morphological filtering: opening removes noise → closing fills holes
        kernel = cv.getStructuringElement(cv.MORPH_RECT, (ksize, ksize))
        mask = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel)
        mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)

        # Contour detection + area/radius filtering
        contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL,
cv.CHAIN_APPROX_SIMPLE)
        targets = []
        for cnt in contours:
            area = cv.contourArea(cnt)
            if area < self.min_target_area:
                continue
            (x, y), radius = cv.minEnclosingCircle(cnt)
            if radius < self.min_target_radius:
                continue
            targets.append((cnt, area, (int(x), int(y)), int(radius)))
        return targets
```

Key logic notes:

- Red is detected by default: `Hmin=0, Hmax=9` covers red at both ends of the HSV hue wheel (0°~9°)
- Morphological filtering order: opening first (remove isolated noise), then closing (fill holes inside the target); the two operations complement each other
- `cv2.RETR_EXTERNAL`: extracts only the outermost contours and ignores nested contours (reduces redundant detections)
- Dual area + radius filtering: `min_target_area=60` filters out pixel-level noise, `min_target_radius=5` filters out line-like artifacts
- S/V upper bounds are kept no lower than 253: after ROI sampling, the saturation/value ranges are stretched automatically for tolerance to lighting changes

2. Key Parameters

> Parameter definition files:
>
> - Launch: `~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_color.launch.py`
> - Node: `~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_color.py`

Parameter	Type	Default Value	Description
`Hmin` / `Hmax`	int	0 / 9	HSV hue range (0~179); red by default
`Smin` / `Smax`	int	85 / 253	HSV saturation range
`Vmin` / `Vmax`	int	126 / 253	HSV value range
`min_target_area`	float	60.0	Minimum target contour area
`min_target_radius`	float	5.0	Minimum enclosing circle radius (px)
`morph_kernel_size`	int	5	Morphological kernel size
`show_binary_window`	bool	false	Whether to show the binary mask side by side

3. Node Description

Node	Function
`detect_color_node`	HSV threshold segmentation + morphological filtering + contour detection + ROI mouse color sampling + rqt dynamic tuning

6. Frequently Asked Questions

Problem	Cause	Solution
Target color not detected	HSV thresholds do not match or lighting changed	Press `r` or drag the mouse to re-select the target color
Too many false detection noise points	Minimum area/radius thresholds too low	Run `ros2 param set /detect_color_node min_target_area 100.0`
Incomplete target region	Insufficient morphological filtering	Increase `morph_kernel_size`